

AIE683 Machine Learning

Week 9: Model Interpretation & Feature Selection

Dr. Hakan Emekci

Hacettepe University, Graduate School of Informatics

A model that learned the wrong lesson

- Hospital data, 1990s: predict the risk of death for pneumonia patients, to decide who can be treated at home.
- Accurate model; one learned rule: **having asthma lowers the risk.**
- True in the data: asthmatic patients were sent straight to intensive care, and the extra care lowered their death rate.
- Deployed as is, the model would have sent the highest-risk patients home.
- (Caruana et al., *Intelligible Models for HealthCare*, KDD 2015)

Other everyday reasons to look inside a model:

- a lender must tell an applicant the main reasons for a rejection;
- a feature turns out to be a leaked copy of the target;
- a domain expert will not trust a model that cannot show what it uses.

Model Interpretation (post-hoc)?

- **Not inference!**
- **Not causality!**
- **Still useful**

Types of explanations

Explain model globally

- How does the output depend on the input?
- Often: some form of marginals

Explain model locally

- Why did it classify this point this way?
- Explanation could look like a "global" one but be different for each point.
- "What is the minimum change to classify it differently?" (counterfactual explanation)

Explaining the model $\neq$ explaining the data

- Model inspection only tells you about the model
- The model might not accurately reflect the data
- Two equally accurate models can tell different stories

Interpretable models as an alternative

- Instead of explaining a black box, fit a model that is readable by construction:
 - sparse linear / logistic models (Week 4)
 - shallow decision trees, rule lists (Week 5)
 - generalized additive models, e.g. Explainable Boosting Machines (`interpret` package)
- On many tabular problems these are within a small margin of gradient boosting.
- Worth trying first when decisions are high-stakes (Rudin, *Nature Machine Intelligence*, 2019).

"Features important to the model"?

Naive:

- `coef_` for linear models (abs value or norm for multi-class)
- `feature_importances_` for tree-based models

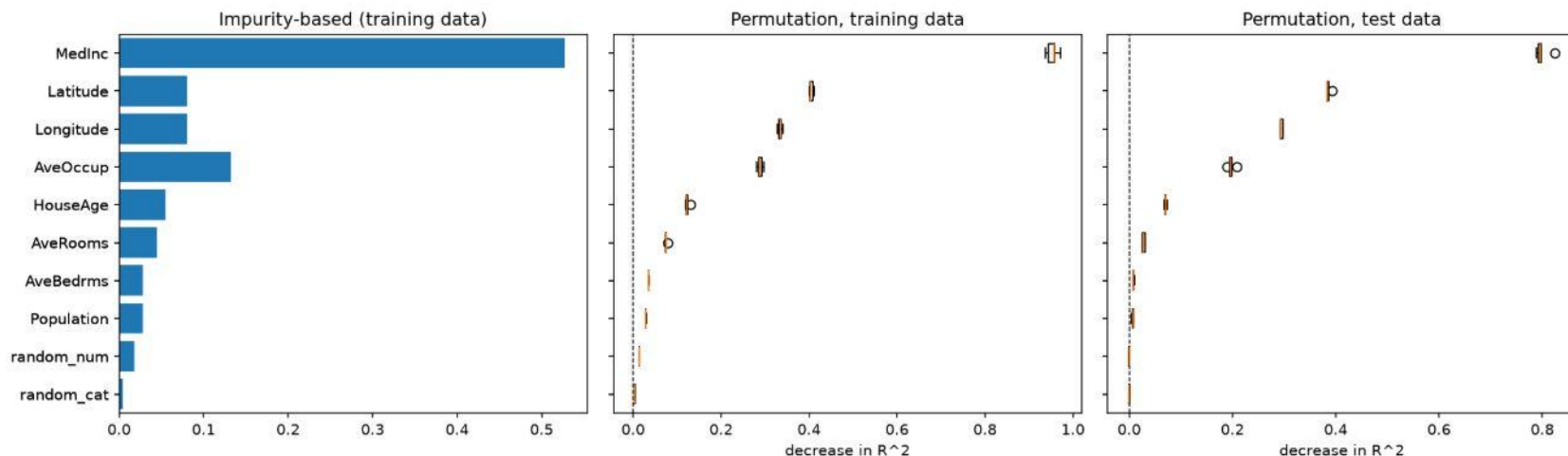
Use with care!

Linear model coefficients

- Relative importance only meaningful after scaling
- Coefficient = change in prediction per unit change in one feature, **holding the others fixed**
- Correlation among features might make coefficients completely uninterpretable
- L1 regularization will pick one at random from a correlated group
- Any penalty will invalidate usual interpretation of linear coefficients

Impurity-based importances: a pitfall

California housing plus two pure-noise columns, random forest (test R^2 0.79)



- Impurity importance is computed on **training** data; it gives `random_num` about as much credit as `Population` and ranks `AveOccup` above `Latitude`.
- Permutation importance on **test** data gives both random columns essentially zero.

Permutation importance

Idea: measure how much the score depends on one feature, without refitting

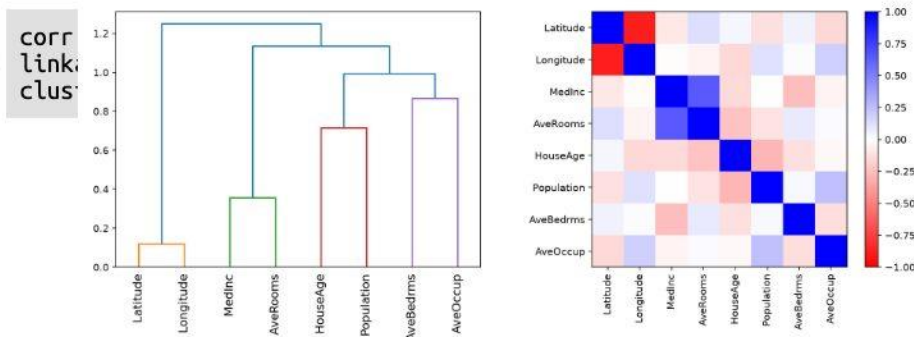
$$I_i^{\text{perm}} = \text{Acc}(f, X, y) - \mathbb{E}_{x_i} [\text{Acc}(f(x_i, X_{-i}), y)]$$

1. Score the fitted model on held-out data.
2. Shuffle column i (breaks its link to y and to the other features), score again.
3. Repeat and average; the drop is the importance of feature i .

```
from sklearn.inspection import permutation_importance
result = permutation_importance(model, X_test, y_test, n_repeats=10,
                               scoring="r2", random_state=0, n_jobs=-1)
result.importances_mean, result.importances_std
```

Permutation importance and correlated features

- Shuffling one of two correlated features creates unrealistic points (6 bedrooms in a 2-room house).
- The model can still use the correlated partner, so **both** can look unimportant.
- Remedy: cluster features by rank correlation, keep one per cluster, then compute importances.



Try it: guess, then measure (10 min)

lab.ipynb, section **Try it 1**

1. Before running anything, write down the three features you think drive California house prices.
2. The lab adds two pure-noise columns and fits a random forest.
3. Compute permutation importance on (a) 3000 training rows and (b) the test set.
4. Compare with `rf.feature_importances_`.

Discuss with your neighbour:

- Did your guess match the measured top 3?
- Which method gives the random columns credit, and why?
- Why do training-set and test-set permutation importances differ?

SHAP: Shapley values for predictions

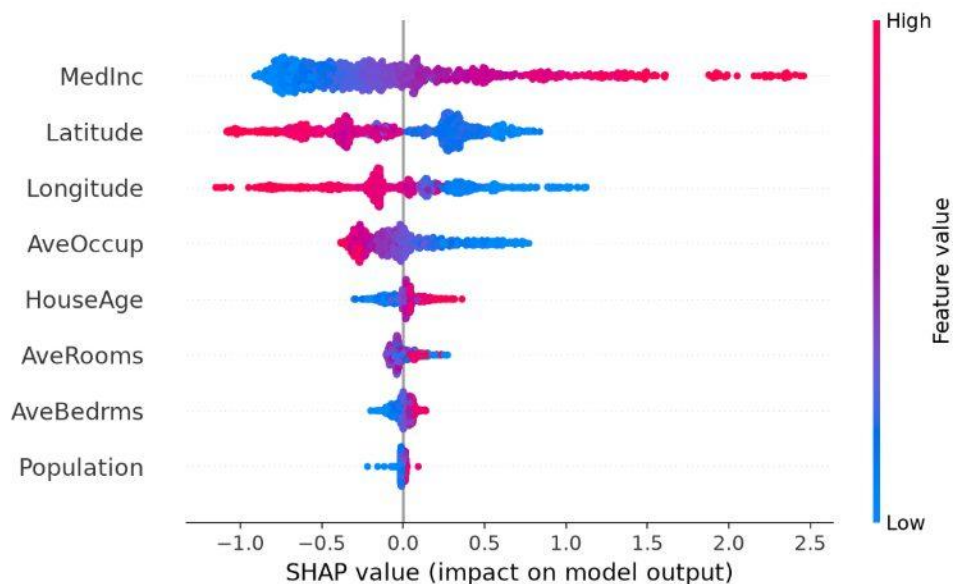
- From cooperative game theory: the features are "players", the prediction is the "payout".
- Feature i gets its average marginal contribution over all subsets S of the other features:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [v(S \cup \{i\}) - v(S)]$$

- $v(S)$: expected model output when only the features in S are known.
- **Additivity**: $f(x) = \phi_0 + \sum_i \phi_i(x)$, where ϕ_0 is the average prediction.
- Exact computation is exponential; fast exact algorithms exist for trees (`TreeExplainer`) and linear models (`LinearExplainer`); sampling approximations otherwise.
- Library: `shap` (<https://github.com/shap/shap>)

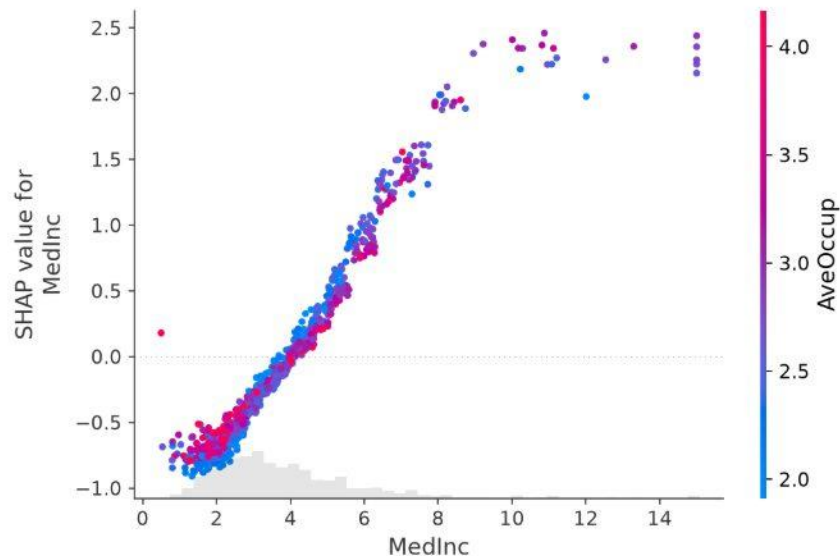
SHAP in practice

```
import shap
explainer = shap.TreeExplainer(gbrt) # gradient boosting, California house
sv = explainer(X_test.sample(1000, random_state=0)) # an Explanation object
shap.plots.beeswarm(sv)
```

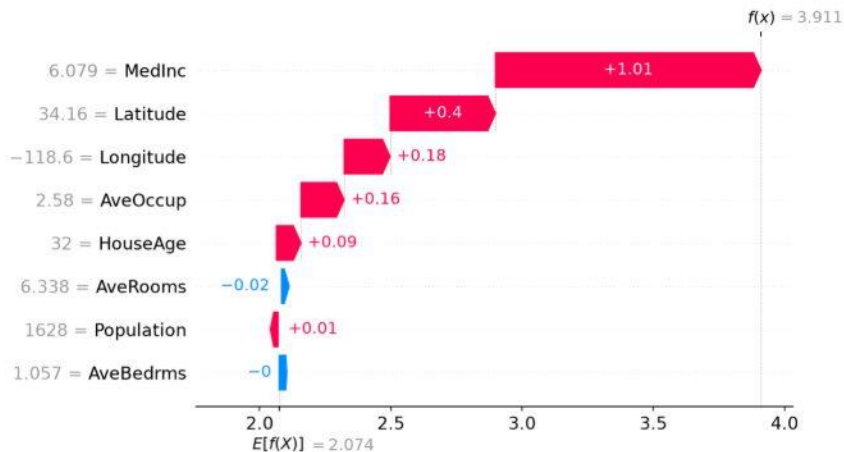


SHAP dependence and local explanations

```
shap.plots.scatter(sv[:, "MedInc"],  
                  color=sv[:, "AveOccup"])
```



```
shap.plots.waterfall(sv[0])
```



What SHAP values do and do not mean

They do

- split one prediction, relative to a baseline, additively among the features;
- describe the **model's** output (in its units: e.g. log-odds for boosted classifiers).

They do not

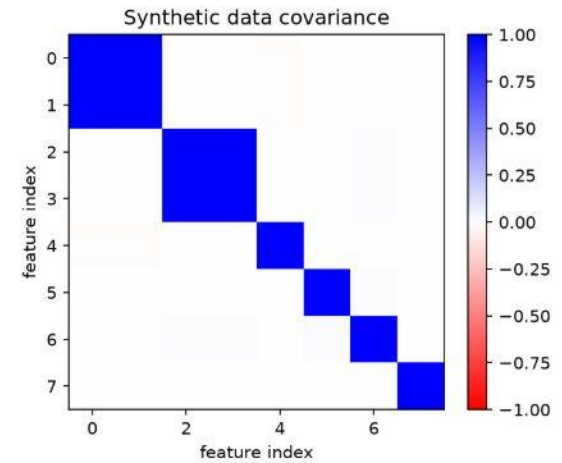
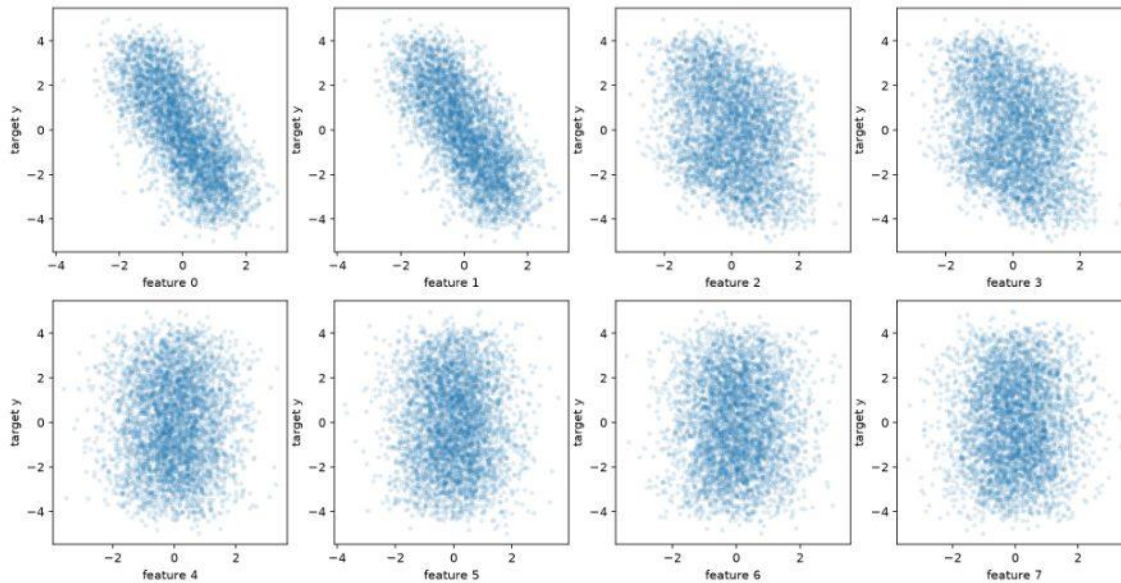
- measure causal effects: "what happens if I raise income" is a different question;
- measure contribution to accuracy: a feature the model overfits can have large SHAP values and zero test importance;
- come with uncertainty or significance;
- have a unique answer with correlated features: `TreeExplainer`'s default (tree-path-dependent) and the interventional version (`feature_perturbation="interventional"` with background data) split credit differently.
- stay fixed when you change the background data or baseline.

Case Study

Toy data

X.shape

(20000, 8)



Models on lots of data

```
lasso = LassoCV().fit(X_train, y_train)
lasso.score(X_test, y_test)
```

0.552

```
ridge = RidgeCV().fit(X_train, y_train)
ridge.score(X_test, y_test)
```

0.552

```
lr = LinearRegression().fit(X_train, y_train)
lr.score(X_test, y_test)
```

0.552

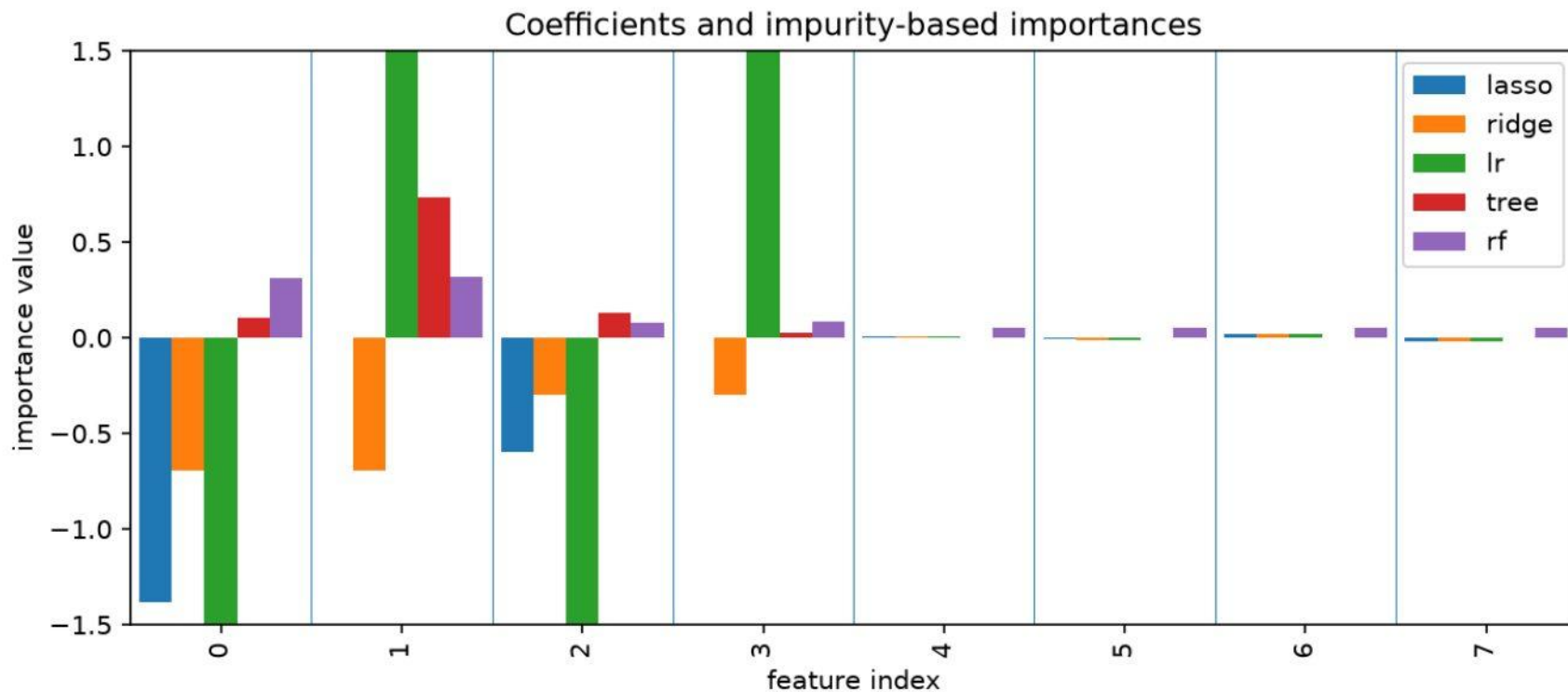
```
param_grid = {'max_leaf_nodes': range(5, 40, 5)}
grid = GridSearchCV(DecisionTreeRegressor(), param_grid, cv=10, n_jobs=-1)
grid.fit(X_train, y_train)
grid.score(X_test, y_test)
```

0.547

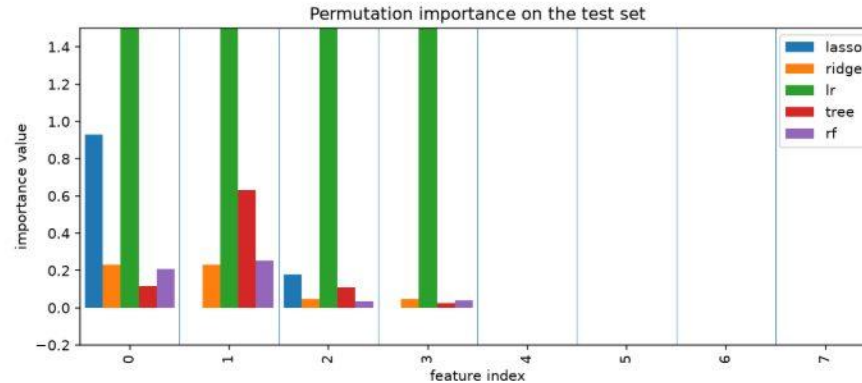
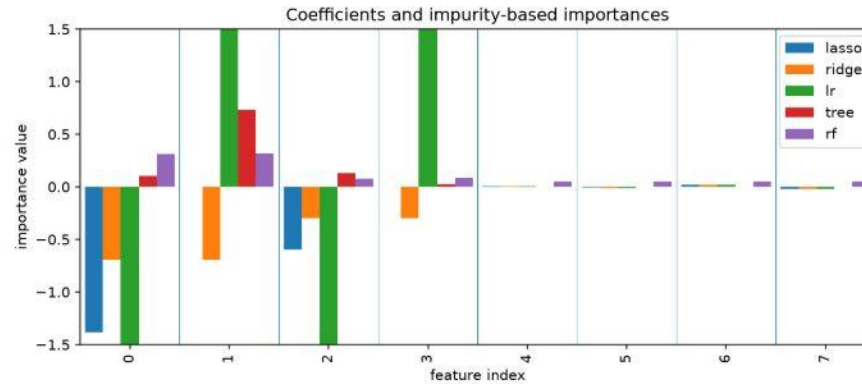
```
rf = RandomForestRegressor(min_samples_leaf=5).fit(X_train, y_train)
rf.score(X_test, y_test)
```

0.537

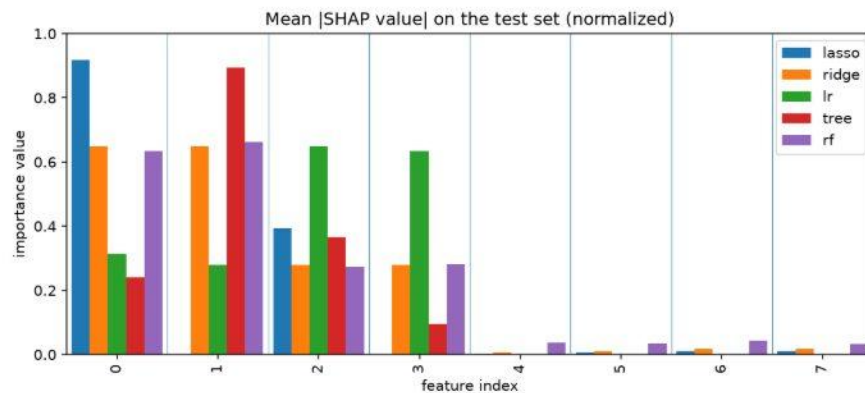
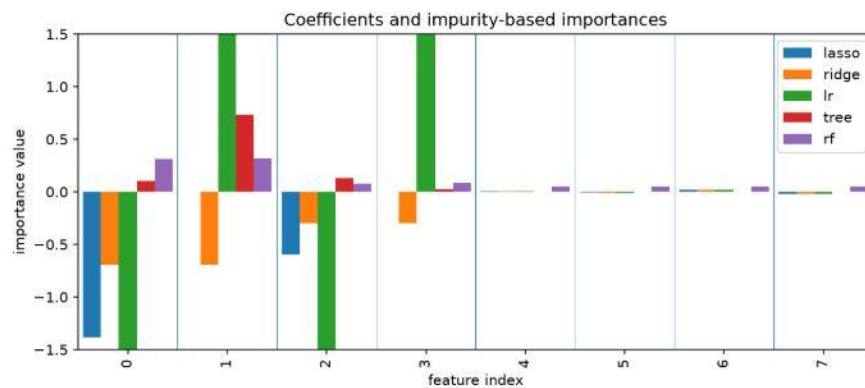
Coefficients and default feature importance



Permutation Importances



SHAP values



More model inspection

Partial Dependence Plots

- Marginal dependence of prediction on one (or two) features

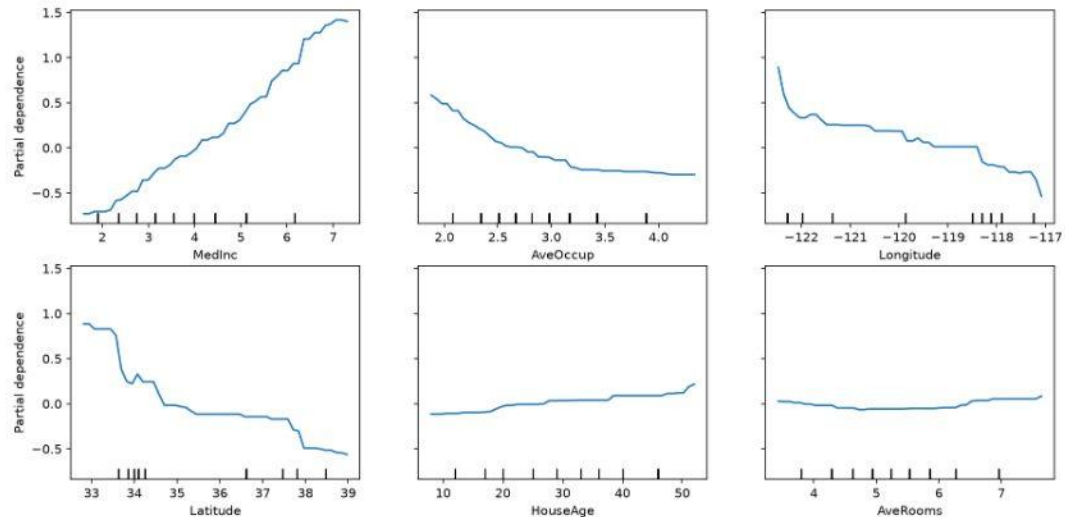
$$f_i^{\text{pdp}}(x_i) = \mathbb{E}_{\mathbf{x}_{-i}} [f(x_i, \mathbf{x}_{-i})]$$

- Process: pick a grid of values for x_i ; for each grid value, set x_i to it in **every** row, predict, and average the predictions.
- Fast recursion method for tree-based models (`method="recursion"`, no data needed)
- For a linear model the PDP is just a straight line with the coefficient as slope.

Partial Dependence

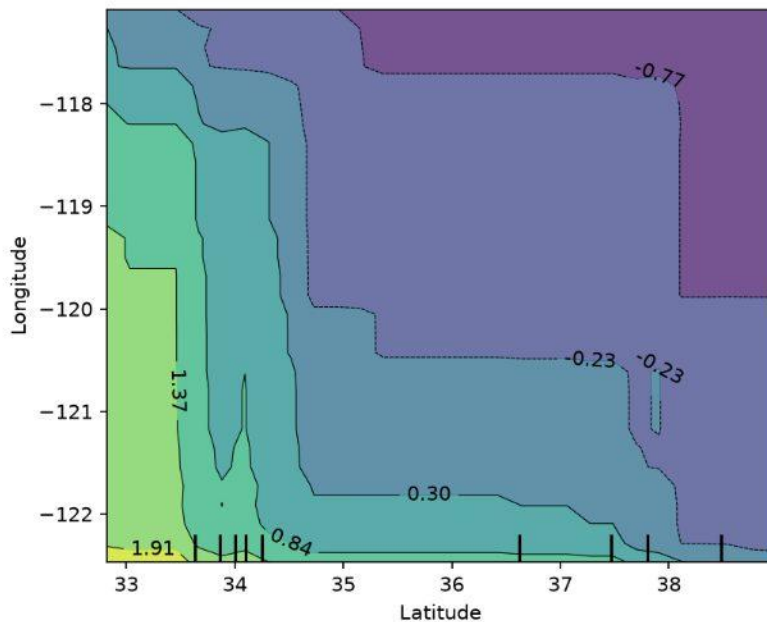
```
from sklearn.inspection import PartialDependenceDisplay
housing = fetch_california_housing(as_frame=True)
X_train, X_test, y_train, y_test = train_test_split(housing.data, housing.target, random_state=0)
gbdt = GradientBoostingRegressor().fit(X_train, y_train)

top6 = X_train.columns[np.argsort(gbdt.feature_importances_)[-6:][::-1]]
PartialDependenceDisplay.from_estimator(gbdt, X_train, top6, n_cols=3, grid_resolution=50)
```

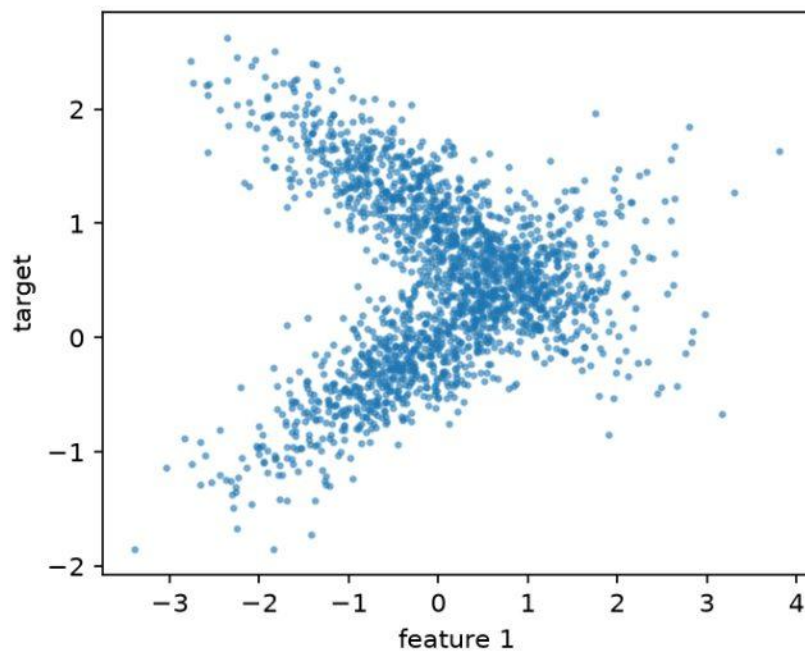
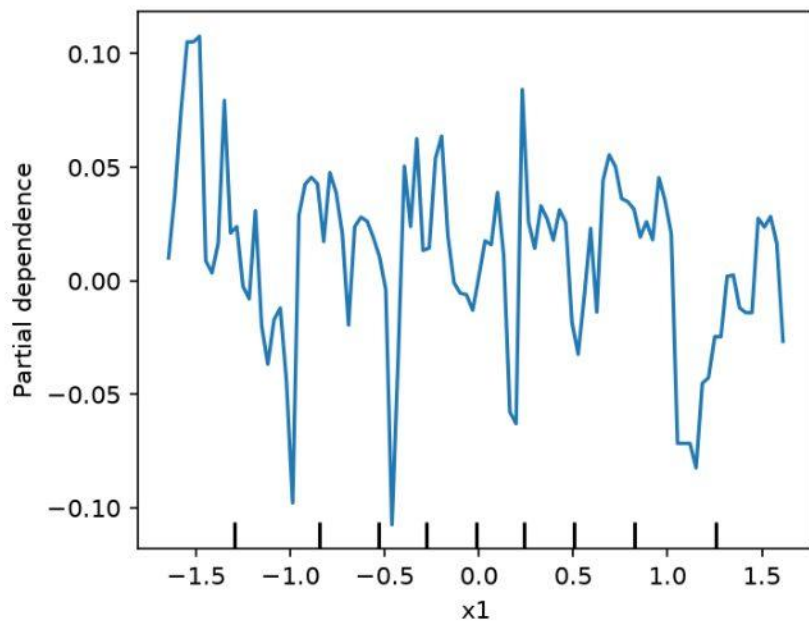


Bivariate Partial Dependence Plots

```
PartialDependenceDisplay.from_estimator(  
    gbrt, X_train, [("Latitude", "Longitude")], grid_resolution=30)
```



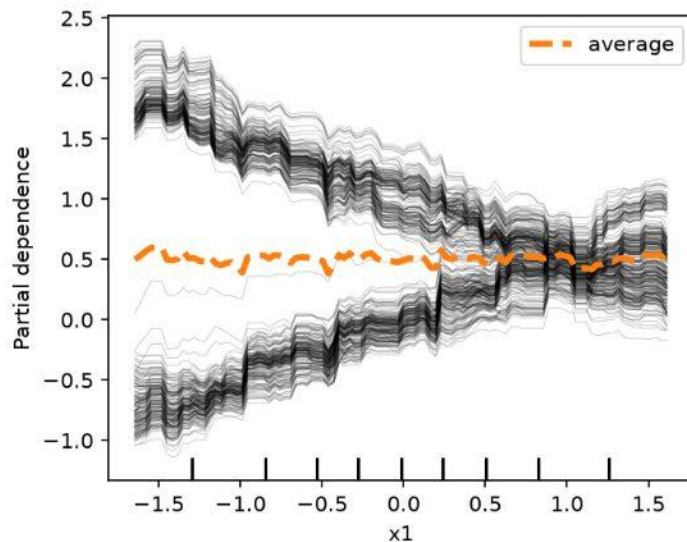
PDP Caveats



Individual Conditional Expectation (ICE)

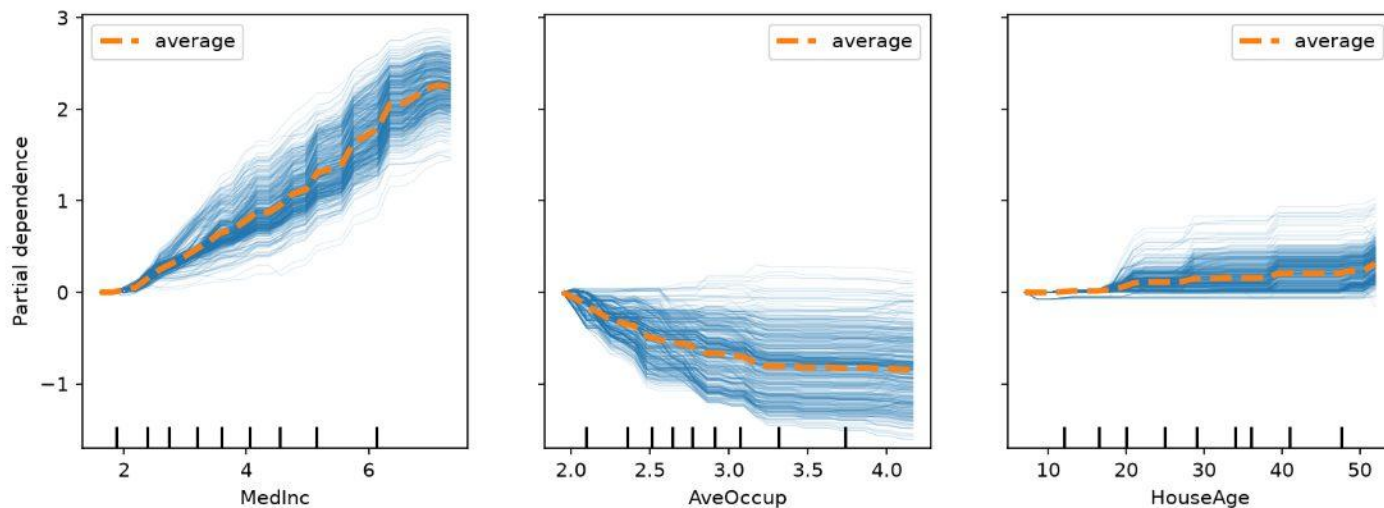
- Like partial dependence plots, without the final averaging: one curve per sample.
- `kind="individual"` or `kind="both"` (ICE + PDP); `centered=True` anchors curves at the left edge.

```
PartialDependenceDisplay.from_estimator(hgb, X, [1], kind="both", subsample=300)
```



ICE on California housing

```
PartialDependenceDisplay.from_estimator(  
    gbdt, X_train.sample(500), ["MedInc", "AveOccup", "HouseAge"],  
    kind="both", centered=True, grid_resolution=30)
```



Feature Selection

Why Select Features?

- Avoid overfitting (?)
- Faster prediction and training
- Less storage for model and dataset
- Cheaper data collection
- More interpretable model

Unsupervised Feature Selection

- May discard important information
- Variance-based: 0 variance or mostly constant
- Covariance-based: remove correlated features
- PCA: remove linear subspaces

Covariance

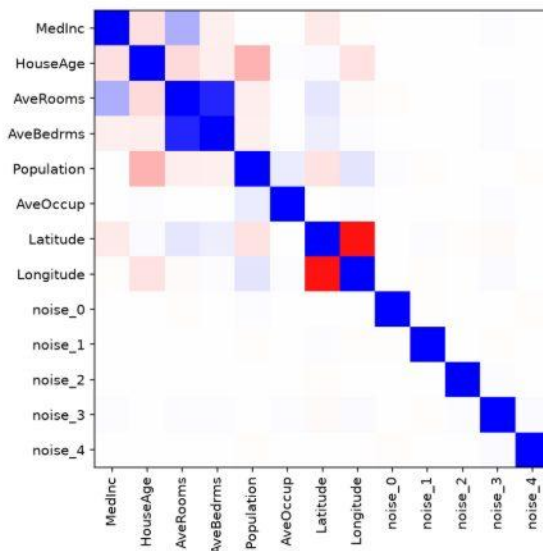
```
from sklearn.preprocessing import scale
```

```
housing = fetch_california_housing(as_frame=True)
```

```
X = housing.data.join(noise) # 8 features + 5 pure-noise columns
```

```
X_train, X_test, y_train, y_test = train_test_split(X, housing.target, random_state
```

```
cov = np.cov(scale(X_train), rowvar=False)
```

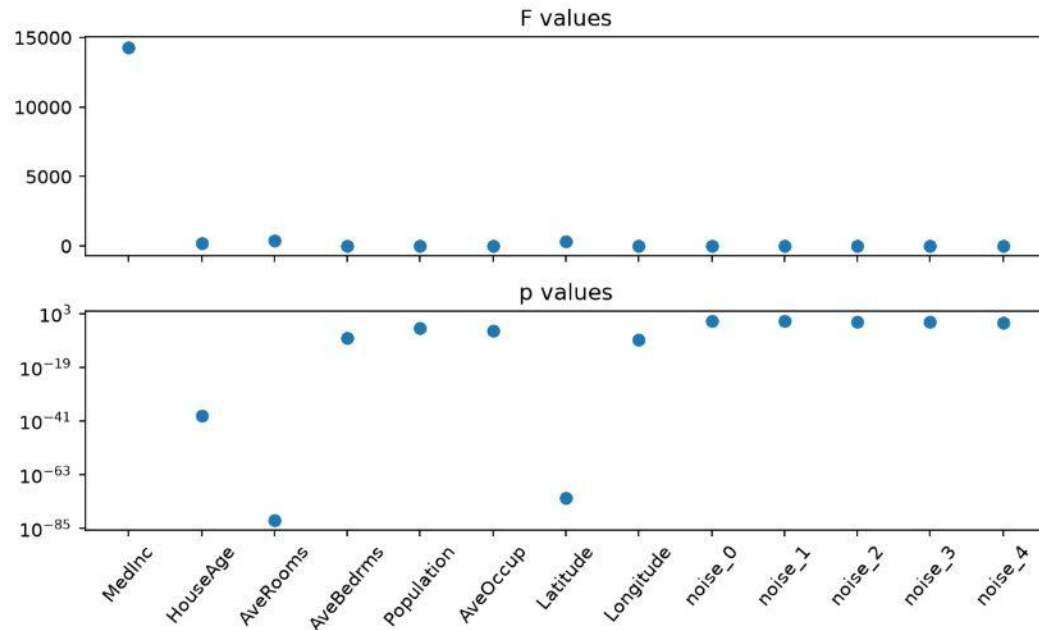


Supervised Feature Selection

Univariate Statistics

- Pick statistic, check p-values!
- `f_regression`, `f_classif`, `chi2`, `mutual_info_*` in scikit-learn

```
from sklearn.feature_selection import f_regression  
f_values, p_values = f_regression(X_train, y_train)
```



```
from sklearn.feature_selection import SelectKBest

select = SelectKBest(k=2, score_func=f_regression).fit(X_train, y_train)
print(X_train.shape, "->", select.transform(X_train).shape)
print(select.get_feature_names_out())
```

```
(15480, 13) -> (15480, 2)
['MedInc' 'AveRooms']
```

```
all_features = make_pipeline(StandardScaler(), RidgeCV())
np.mean(cross_val_score(all_features, X_train, y_train, cv=cv))
```

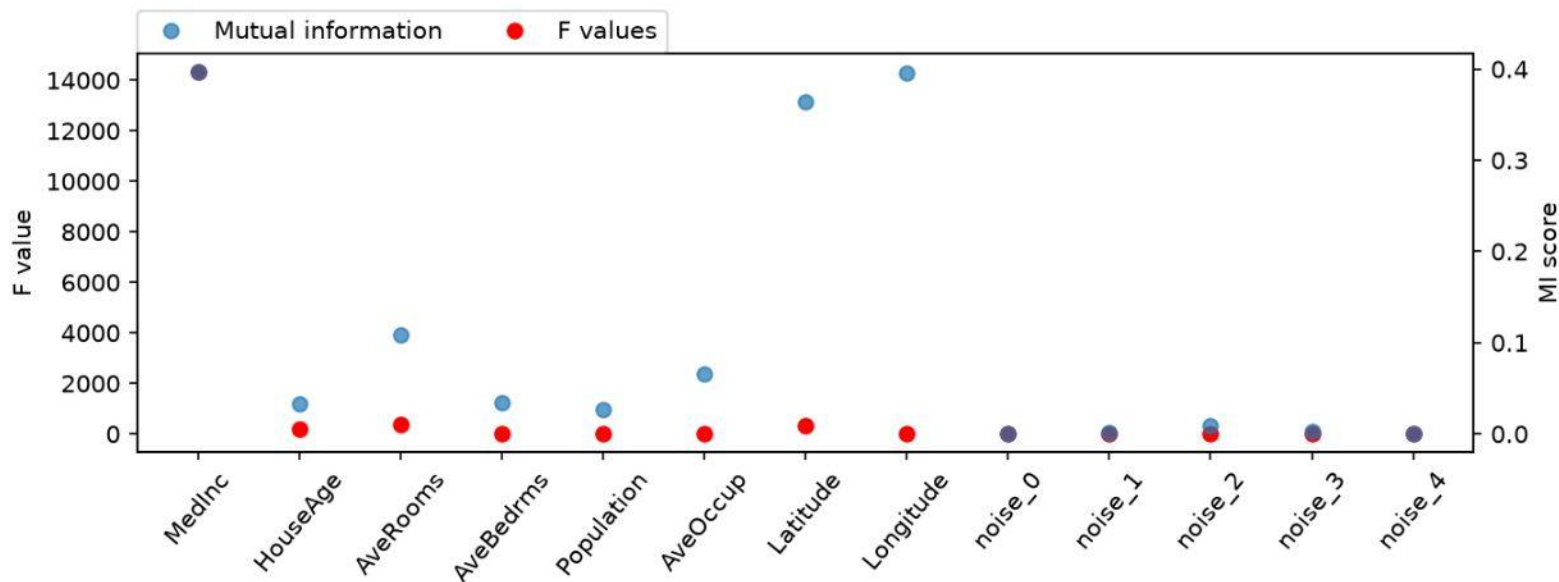
0.606

```
select_2 = make_pipeline(StandardScaler(),
                        SelectKBest(k=2, score_func=f_regression), RidgeCV())
np.mean(cross_val_score(select_2, X_train, y_train, cv=cv))
```

0.483

Mutual Information

```
from sklearn.feature_selection import mutual_info_regression  
scores = mutual_info_regression(X_train, y_train)
```



Model-Based Feature Selection

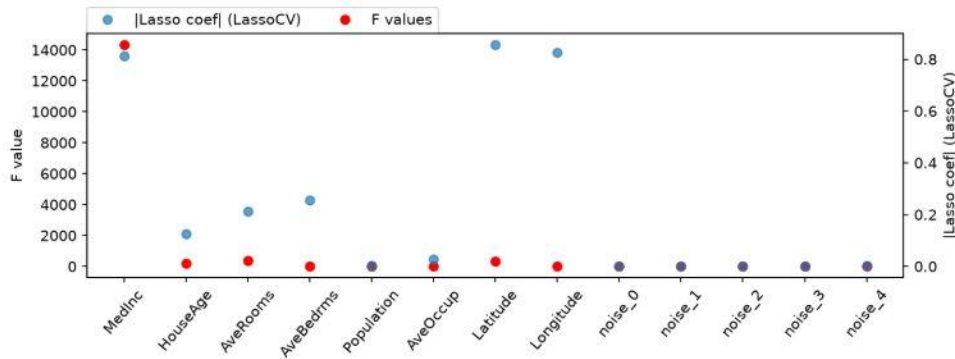
- Get best fit for a particular model
- Ideally: exhaustive search over all possible combinations
- Exhaustive is infeasible (and has multiple testing issues)
- Use heuristics in practice.

Model based (single fit)

- Build a model, select "features important to model"
- Lasso, other linear models, tree-based Models
- Multivariate - linear models assume linear relation

```
from sklearn.linear_model import LassoCV
X_train_scaled = scale(X_train)
lasso = LassoCV(cv=cv).fit(X_train_scaled, y_train)
print(lasso.coef_)
```

```
[ 0.814  0.124 -0.212  0.255 -0.004 -0.027 -0.855 -0.825  0.    -0.
  0.    -0.001 -0.004]
```



SelectFromModel

```
from sklearn.feature_selection import SelectFromModel
select_lassocv = SelectFromModel(LassoCV(cv=cv), threshold=1e-5)
pipe_lassocv = make_pipeline(StandardScaler(), select_lassocv, RidgeCV())
pipe_lassocv.fit(X_train, y_train)[: -1].get_feature_names_out()
```

```
['MedInc' 'HouseAge' 'AveRooms' 'AveBedrms' 'Population' 'AveOccup'
 'Latitude' 'Longitude' 'noise_3' 'noise_4']
```

```
np.mean(cross_val_score(pipe_lassocv, X_train, y_train, cv=cv))
```

0.606

```
select_lasso = SelectFromModel(Lasso(alpha=0.1)) # could grid-search alpha
pipe_lasso = make_pipeline(StandardScaler(), select_lasso, RidgeCV())
np.mean(cross_val_score(pipe_lasso, X_train, y_train, cv=cv))
```

0.524

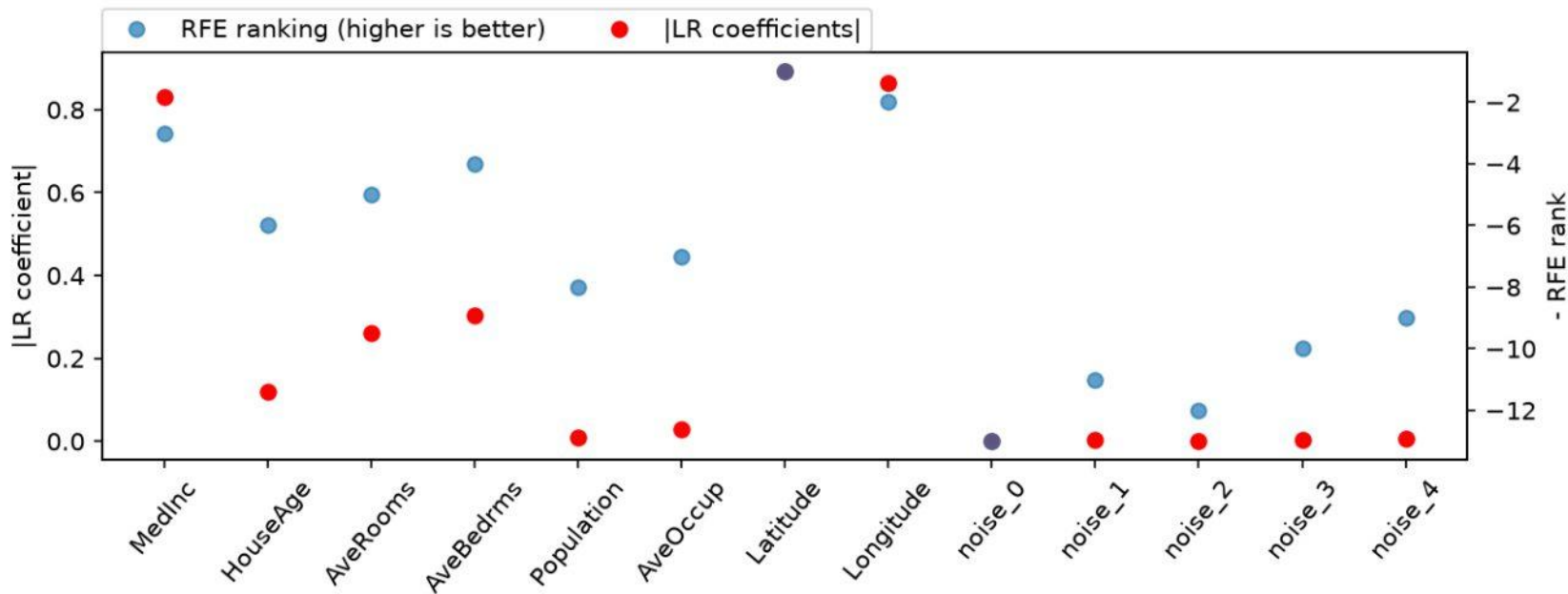
Recursive Feature Elimination

- Uses feature importances / coefficients, similar to `SelectFromModel`
- Iteratively removes features (one by one or in groups)
- Runtime: $(n_{\text{features}} - n_{\text{features_to_keep}}) / \text{step}$ model fits

```
from sklearn.feature_selection import RFE
```

```
# create a ranking of all features by eliminating down to one  
rfe = RFE(LinearRegression(), n_features_to_select=1)  
rfe.fit(X_train_scaled, y_train)  
rfe.ranking_
```

```
array([ 3,  6,  5,  4,  8,  7,  1,  2, 13, 11, 12, 10,  9])
```



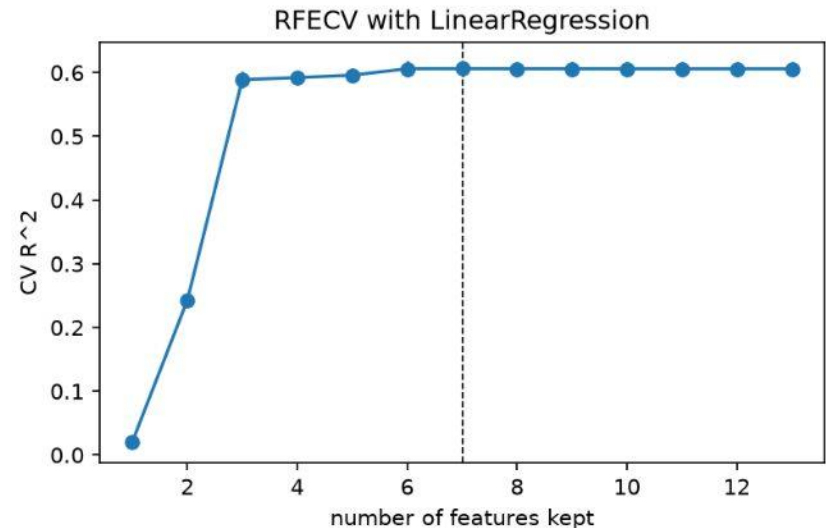
RFECV

```
from sklearn.feature_selection import RFECV
rfecv = RFECV(LinearRegression(), cv=cv).fit(X_train_scaled, y_train)
feature_names[rfecv.support_]
```

```
['MedInc' 'HouseAge' 'AveRooms' 'AveBedrms' 'AveOccup' 'Latitude' 'Longitude']
```

```
pipe = make_pipeline(StandardScaler(),
                     RFECV(LinearRegression(), cv=cv),
                     RidgeCV())
np.mean(cross_val_score(pipe, X_train, y_train, cv=cv))
```

0.602



```
from sklearn.preprocessing import PolynomialFeatures, QuantileTransformer
pipe_poly_rfe = make_pipeline(
    QuantileTransformer(output_distribution="normal"), # tame heavy tails first
    PolynomialFeatures(), StandardScaler(),
    RFECV(LinearRegression(), step=5, cv=cv), RidgeCV())
np.mean(cross_val_score(pipe_poly_rfe, X_train, y_train, cv=cv))
```

0.701

```
pipe_poly_rfe.fit(X_train, y_train)
pipe_poly_rfe.named_steps["rfecv"].n_features_
```

40 # of 105 polynomial features (can vary slightly: near-ties)

Wrapper Methods

- Can be applied for ANY model!
- Shrink / grow feature set by greedy search
- Called Forward or Backward selection
- Run CV / train-val split per candidate feature
- Complexity: about $n_{\text{features}} * (n_{\text{features}} + 1) / 2$ CV runs
- `SequentialFeatureSelector` in scikit-learn

SequentialFeatureSelector

```
from sklearn.feature_selection import SequentialFeatureSelector
sfs = SequentialFeatureSelector(LinearRegression(), n_features_to_select=7,
                               direction="backward", cv=cv)
sfs.fit(X_train_scaled, y_train)
feature_names[sfs.get_support()]
```

```
['MedInc' 'HouseAge' 'AveRooms' 'AveBedrms' 'AveOccup' 'Latitude' 'Longitude']
```

```
sfs = SequentialFeatureSelector(LinearRegression(), n_features_to_select="auto",
                               tol=1e-3, direction="forward", cv=cv)
```

```
['MedInc' 'HouseAge' 'AveRooms' 'AveBedrms' 'Latitude' 'Longitude']
```

What goes wrong: selecting features outside CV

Pure noise: 100 samples, 5000 random features, random labels. True accuracy: 50%.

```
X_sel = SelectKBest(k=20).fit_transform(X_noise, y_noise) # uses ALL labels
cross_val_score(LogisticRegression(), X_sel, y_noise, cv=5).mean()
```

0.85

```
pipe = make_pipeline(SelectKBest(k=20), LogisticRegression()) # refit per fold
cross_val_score(pipe, X_noise, y_noise, cv=5).mean()
```

0.48

- Classic mistake in wide data (genomics, text features); published results have been retracted for it (Ambroise & McLachlan, PNAS 2002).

Try it: find the leak (10 min)

lab.ipynb, section Try it 2

1. Run the provided code: it reports a high cross-validated accuracy on pure noise.
2. In one sentence, explain why the estimate is wrong.
3. Build `honest_pipe` so that selection is refit inside every fold, and report its score.
4. Bonus: change `k` and the number of noise features. How does the leaky score change?

Tracking interpretation results with MLflow

```
mlflow.set_experiment("week09-interpretation")
for name, model in candidates.items():
    with mlflow.start_run(run_name=name):
        model.fit(X_train, y_train)
        mlflow.log_metric("test_r2", model.score(X_test, y_test))
        pi = permutation_importance(model, X_test, y_test, n_repeats=5)
        mlflow.log_table(pi_df, artifact_file="permutation_importance.json")
        mlflow.log_figure(fig, "permutation_importance.png") # plot as artifact
        if name != "ridge":
            shap.plots.beeswarm(shap.TreeExplainer(model)(X_shap), show=False)
            mlflow.log_figure(plt.gcf(), "shap_beeswarm.png")
```

- Each run stores its score **and** its explanation; compare them side by side in `mlflow ui`.
- Log the selected feature list (`mlflow.log_dict`) for feature-selection runs.

Try it: feature-budget challenge (15 min)

lab.ipynb, section Try it 3

- Data: California housing + 5 noise columns (13 features), 5000-row development sample.
- Goal: highest 5-fold CV R^2 using **at most 3 features**.
- Any model, any selection method: manual, univariate, `SelectFromModel`, RFE, SFS.
- `check_pipeline(my_pipeline)` reports your CV score and counts your features.
- At the end, every team's pipeline is evaluated **once** on the test set.

Summary

- Explaining a model is not explaining the data, and not causal inference.
- Prefer **permutation importance on held-out data** to `feature_importances_`.
- Correlated features share or hide importance in every method; cluster them first.
- PDPs show average effects; **ICE** curves (`kind="both"`) reveal heterogeneity.
- **SHAP** gives additive local explanations of the model's output; read the caveats.
- Feature selection: univariate (fast, blind to interactions), `SelectFromModel`, `RFECV`, `SequentialFeatureSelector`; always **inside** the CV pipeline.
- Next week: unsupervised learning (PCA, t-SNE/UMAP, clustering, mixture models).

Questions?

Backup

Drop-column importance

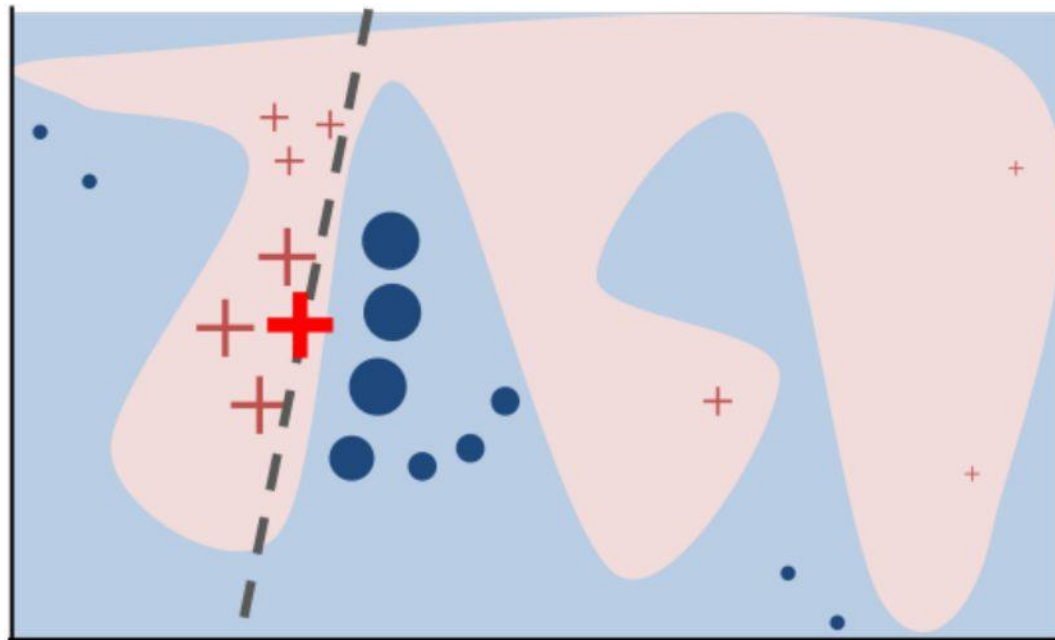
$$I_i^{\text{drop}} = \text{Acc}(f, X, y) - \text{Acc}(f', X_{-i}, y)$$

```
def drop_feature_importance(est, X, y):  
    base_score = np.mean(cross_val_score(est, X, y))  
    scores = []  
    for feature in range(X.shape[1]):  
        mask = np.ones(X.shape[1], dtype=bool)  
        mask[feature] = False  
        this_score = np.mean(cross_val_score(est, X[:, mask], y))  
        scores.append(base_score - this_score)  
    return np.array(scores)
```

- Doesn't really explain the model (refits for each feature)
- Can't deal with correlated features well
- Very slow
- Can be used for feature selection

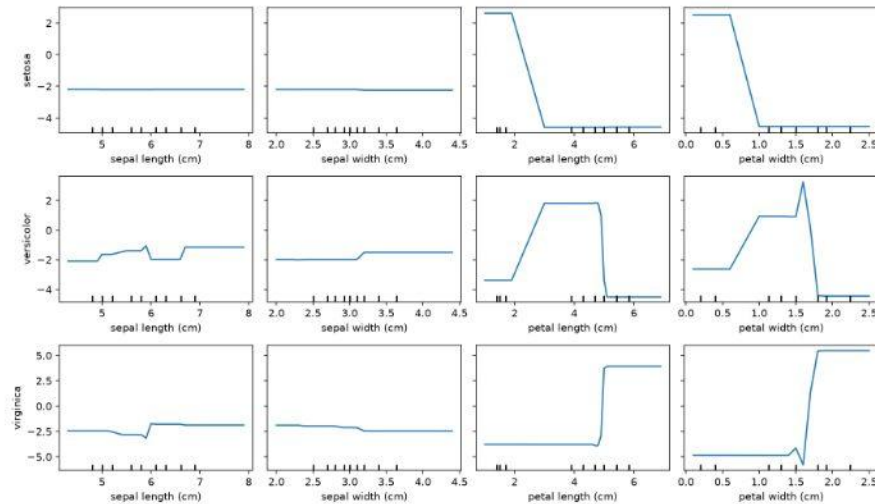
LIME

- Fit a sparse linear model to the black box's predictions in a neighbourhood of one point.
- Explains a single prediction locally.
- Paper: "Why Should I Trust You?" Explaining the Predictions of Any Classifier (2016)
- Package: `lime`. Explanations depend on the kernel width and the random sampling, and can change between runs; SHAP has largely replaced it in tabular practice.



Partial Dependence for Classification

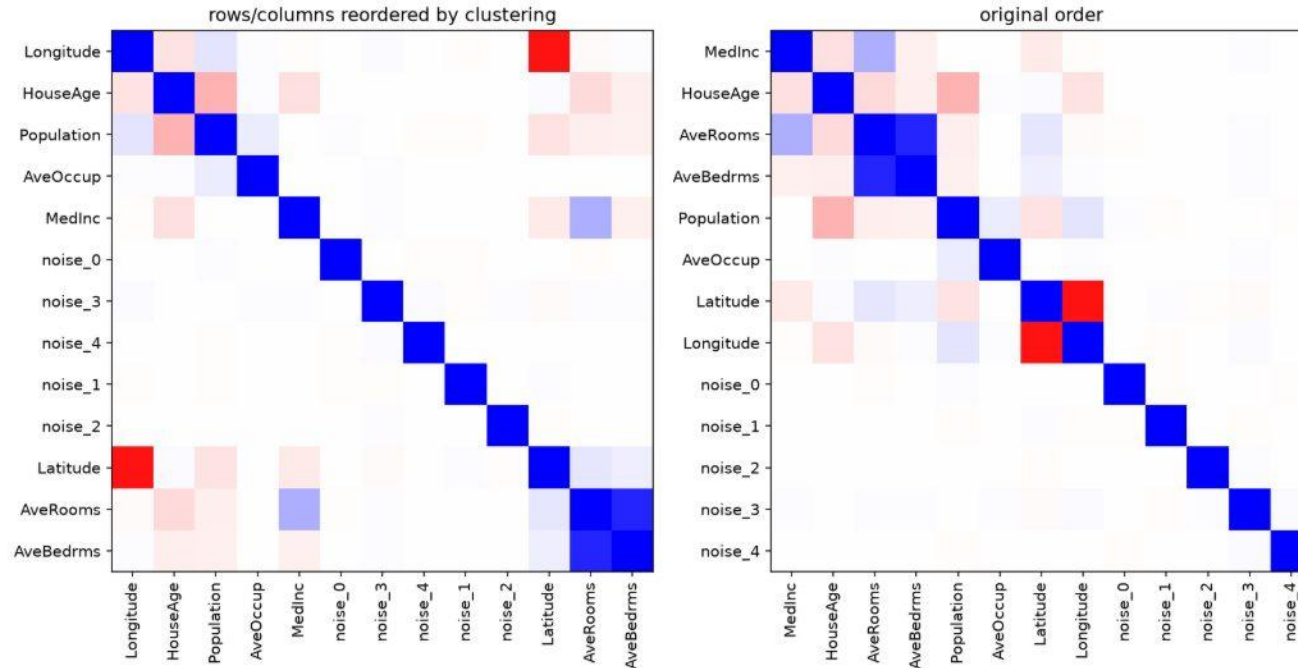
```
for label in range(3):  
    PartialDependenceDisplay.from_estimator(  
        gbc, X_train, range(4), target=label, feature_names=iris.feature_names,  
        grid_resolution=50, ax=axes[label])
```



Types of Feature Selection

- Unsupervised vs Supervised
- Univariate vs Multivariate
- Model based or not

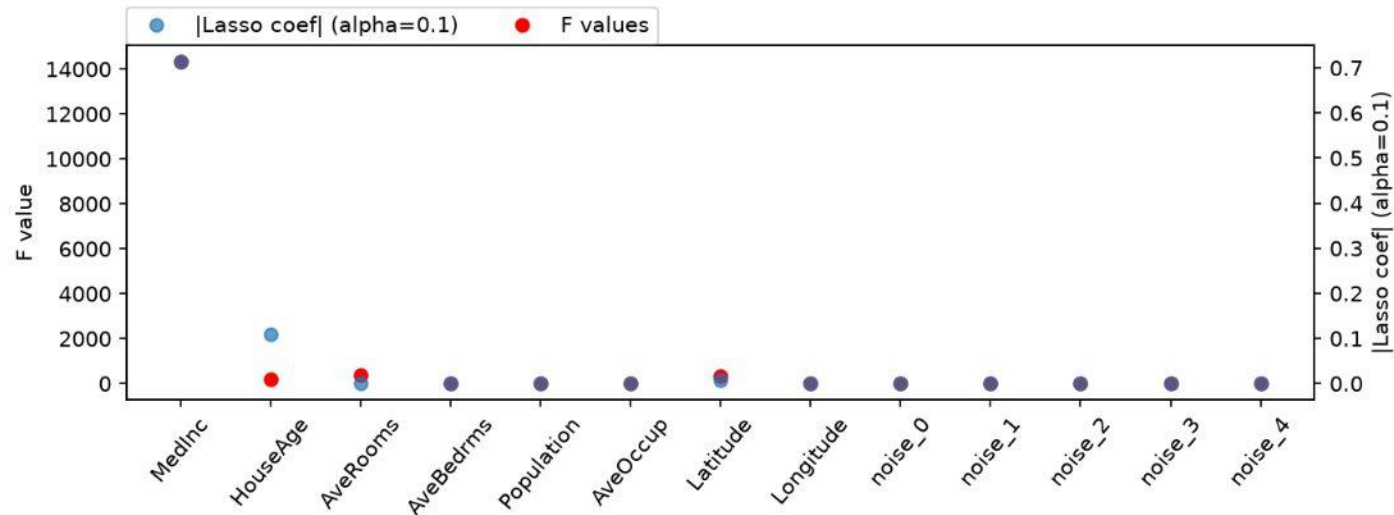
```
from scipy.cluster import hierarchy
order = np.array(hierarchy.dendrogram(
    hierarchy.ward(cov), no_plot=True)['ivl'], dtype="int")
```



Changing Lasso alpha

```
from sklearn.linear_model import Lasso
lasso = Lasso(alpha=0.1).fit(X_train_scaled, y_train)
print(lasso.coef_)
```

```
[ 0.713  0.109 -0.      0.      -0.      -0.      -0.008 -0.      0.      0.
  0.      -0.      -0.      ]
```



Iterative Model-Based Selection

- Fit model, find least important feature, remove, iterate.
- Or: Start with single feature, find most important feature, add, iterate.